

Import Libraries

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dr
from tensorflow.keras.utils import to_categorical
```

```
from google.colab import files
upload = files.upload()
```

[Choose Files](#) doctor_han...g_labels.csv
doctor_handwriting_labels.csv(text/csv) - 3016 bytes, last modified: 12/6/2025 - 100% done
Saving doctor_handwriting_labels.csv to doctor_handwriting_labels.csv

Load CSV File

```
import pandas as pd
df = pd.read_csv('doctor_handwriting_labels.csv')
```

```
df.head()
```

	filename	label	
0	.trashed-1730802496-IMG_20241006_151957 - Copy...	cefiget 40mg	
1	IMG_20241006_141949 - Copy.jpg	tab azimax	
2	IMG_20241006_142328 - Copy.jpg	tab toniflex	
3	IMG_20241006_142533 - Copy.jpg	nims	
4	IMG_20241006_142757 - Copy.jpg	dicloran	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

Unzip Image Dataset (FIXED)

```
!unzip img.zip -d /content/img
```

```
inflating: /content/img/img/IMG_20241006_142533 - Copy.jpg
inflating: /content/img/img/IMG_20241006_142757 - Copy.jpg
inflating: /content/img/img/IMG_20241006_142934 - Copy.jpg
inflating: /content/img/img/IMG_20241006_143132 - Copy.jpg
inflating: /content/img/img/IMG_20241006_143429 - Copy.jpg
inflating: /content/img/img/IMG_20241006_143544 - Copy.jpg
inflating: /content/img/img/IMG_20241006_143810 - Copy.jpg
inflating: /content/img/img/IMG_20241006_143853 - Copy.jpg
inflating: /content/img/img/IMG_20241006_144119 - Copy.jpg
inflating: /content/img/img/IMG_20241006_144252 - Copy.jpg
inflating: /content/img/img/IMG_20241006_144408 - Copy.jpg
inflating: /content/img/img/IMG_20241006_144505 - Copy.jpg
inflating: /content/img/img/IMG_20241006_144555.jpg
inflating: /content/img/img/IMG_20241006_144641.jpg
inflating: /content/img/img/IMG_20241006_144753.jpg
inflating: /content/img/img/IMG_20241006_144902.jpg
inflating: /content/img/img/IMG_20241006_144955.jpg
inflating: /content/img/img/IMG_20241006_145056.jpg
inflating: /content/img/img/IMG_20241006_145141.jpg
inflating: /content/img/img/IMG_20241006_145250 - Copy.jpg
inflating: /content/img/img/IMG_20241006_145250.jpg
inflating: /content/img/img/IMG_20241006_145439.jpg
inflating: /content/img/img/IMG_20241006_145636.jpg
inflating: /content/img/img/IMG_20241006_145902.jpg
inflating: /content/img/img/IMG_20241006_150012 - Copy.jpg
inflating: /content/img/img/IMG_20241006_150012.jpg
inflating: /content/img/img/IMG_20241006_150045.jpg
inflating: /content/img/img/IMG_20241006_150130.jpg
inflating: /content/img/img/IMG_20241006_150224.jpg
inflating: /content/img/img/IMG_20241006_150321.jpg
inflating: /content/img/img/IMG_20241006_150507.jpg
inflating: /content/img/img/IMG_20241006_150800.jpg
inflating: /content/img/img/IMG_20241006_150836.jpg
inflating: /content/img/img/IMG_20241006_150902.jpg
inflating: /content/img/img/IMG_20241006_150946.jpg
inflating: /content/img/img/IMG_20241006_151057.jpg
inflating: /content/img/img/IMG_20241006_151126.jpg
inflating: /content/img/img/IMG_20241006_151222.jpg
inflating: /content/img/img/IMG_20241006_151517.jpg
inflating: /content/img/img/IMG_20241006_151556.jpg
inflating: /content/img/img/IMG_20241006_151638.jpg
inflating: /content/img/img/IMG_20241006_151719.jpg
inflating: /content/img/img/IMG_20241006_151807.jpg
inflating: /content/img/img/IMG_20241006_151901.jpg
inflating: /content/img/img/IMG_20241006_152046.jpg
inflating: /content/img/img/IMG_20241006_152156.jpg
inflating: /content/img/img/IMG_20241006_152333.jpg
inflating: /content/img/img/IMG_20241006_152453.jpg
inflating: /content/img/img/IMG_20241006_152643.jpg
inflating: /content/img/img/IMG_20241006_152937.jpg
inflating: /content/img/img/IMG_20241006_153027.jpg
inflating: /content/img/img/IMG_20241006_153118.jpg
inflating: /content/img/img/IMG_20241006_153145.jpg
inflating: /content/img/img/IMG_20241006_153240.jpg
inflating: /content/img/img/IMG_20241006_153705.jpg
inflating: /content/img/img/IMG_20241006_153802.jpg
inflating: /content/img/img/IMG_20241006_153855.jpg
```

Encode Labels

```
labels = df['label'].unique()
label_map = {label: i for i, label in enumerate(labels)}

df['label_encoded'] = df['label'].map(label_map)

label_map
```

```
'nims': 3,
'dicloran': 4,
'breeky': 5,
'pronaz': 6,
'movax': 7,
'cap caricef': 8,
'bisleri': 9,
'tab novidat': 10,
'tab azitma 500mg': 11,
'tab tonoflex p': 12,
'tab nims 100mg': 13,
'tab dicloran': 14,
'tab breaky': 15,
'tab provas': 16,
'cap caricef 40mg': 17,
'tab bisleri': 18,
'tab distalgesic': 19,
'tab atcomid': 20,
'tab atconate': 21,
'tab mesulid': 22,
'movelate': 23,
'uriguard': 24,
'na dard': 25,
'tab atcam': 26,
'tab my d': 27,
'tab ostium': 28,
'tab distalgasic': 29,
'tab movalate': 30,
'tab uriguard': 31,
'tab na dard': 32,
'tab getryl 1mg': 33,
'tab covam 5-80mg': 34,
'tab fexet 60mg': 35,
'starcoc 90': 36,
'leflor 200mg': 37,
'tab lipiget 10mg': 38,
'cap risek 20mg': 39,
'nebil ': 40,
'tab montiget 10mg': 41,
'montiget': 42,
'getryl': 43,
'covam': 44,
'fexet': 45,
'leflor': 46,
'risek': 47,
```

```
novitate': 51,
'fixit': 52,
'starcox': 53,
'cefeget': 54,
'distalgesic': 55,
'tab movelate': 56,
'my-d': 57,
'ostium': 58,
'provas': 59,
'caricef': 60}
```

Load & Preprocess Images

```
IMAGE_DIR = "/content/img/img"
IMG_SIZE = 128

X = []
y = []

for _, row in df.iterrows():
    img_path = os.path.join(IMAGE_DIR, row['filename'])
    img = load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
    img = img_to_array(img) / 255.0

    X.append(img)
    y.append(row['label_encoded'])

X = np.array(X)
y = to_categorical(y)
```

Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, y_train.shape)
```

```
(71, 128, 128, 3) (71, 61)
```

Build CNN Model

```
input_tensor = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))

x = Conv2D(32, (3,3), activation='relu')(input_tensor)
x = MaxPooling2D(2,2)(x)

x = Conv2D(64, (3,3), activation='relu')(x)
x = MaxPooling2D(2,2)(x)
```

```

x = Flatten()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output_tensor = Dense(len(label_map), activation='softmax')(x)

model = tf.keras.Model(inputs=input_tensor, outputs=output_tensor)

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 128, 128, 3)	0
conv2d_2 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_3 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 30, 30, 64)	0
flatten_1 (Flatten)	(None, 57600)	0
dense_2 (Dense)	(None, 128)	7,372,928
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 61)	7,869

Total params: 7,400,189 (28.23 MB)
 Trainable params: 7,400,189 (28.23 MB)
 Non-trainable params: 0 (0.00 B)

Train the Model

```

history = model.fit(
    X_train, y_train,
    epochs=10,
    validation_split=0.1,
    batch_size=32
)

```

Epoch 1/10
 2/2 ————— 4s 1s/step - accuracy: 0.0212 - loss: 6.1137 - val_a
 Epoch 2/10
 2/2 ————— 2s 1s/step - accuracy: 0.0630 - loss: 6.3202 - val_a
 Epoch 3/10

```

2/2 ————— 2s 864ms/step - accuracy: 0.0630 - loss: 4.6392 - va
Epoch 4/10
2/2 ————— 3s 873ms/step - accuracy: 0.0000e+00 - loss: 4.1409
Epoch 5/10
2/2 ————— 3s 849ms/step - accuracy: 0.0422 - loss: 4.1618 - va
Epoch 6/10
2/2 ————— 3s 841ms/step - accuracy: 0.0316 - loss: 4.0828 - va
Epoch 7/10
2/2 ————— 3s 1s/step - accuracy: 0.0210 - loss: 4.1124 - val_a
Epoch 8/10
2/2 ————— 2s 834ms/step - accuracy: 0.0106 - loss: 4.0786 - va
Epoch 9/10
2/2 ————— 2s 870ms/step - accuracy: 0.0210 - loss: 4.1074 - va
Epoch 10/10
2/2 ————— 3s 820ms/step - accuracy: 0.0212 - loss: 4.0809 - va

```

Evaluate Model

```

loss, acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", acc)

```

```

1/1 ————— 0s 155ms/step - accuracy: 0.0000e+00 - loss: 4.1346
Test Accuracy: 0.0

```

Explainable AI (Grad-CAM)

Grad-CAM Implementation

```

def grad_cam(model, img_array, last_conv_layer_name):
    grad_model = tf.keras.models.Model(
        [model.inputs],
        [model.get_layer(last_conv_layer_name).output, model.output]
    )

    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        loss = predictions[:, np.argmax(predictions[0])]

    grads = tape.gradient(loss, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0,1,2))

    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)

    heatmap = np.maximum(heatmap, 0) / np.max(heatmap)
    return heatmap

```

Visualize Explanation

```
img = X_test[0]
img_array = np.expand_dims(img, axis=0)

_ = model(img_array) # Explicitly call the model to build its graph and defi

heatmap = grad_cam(model, img_array, 'conv2d_3')

plt.imshow(img)
plt.imshow(heatmap, cmap='jet', alpha=0.5)
plt.axis('off')
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: U
Expected: [['keras_tensor_9']]
Received: inputs=Tensor(shape=(1, 128, 128, 3))
  warnings.warn(msg)
(np.float64(-0.5), np.float64(60.5), np.float64(60.5), np.float64(-0.5))
```



